

REMOTE_DEPLOYMENT

Remote Deployment Guide

This guide explains how to deploy containers to remote hosts using the Containers module's remote distribution feature.

Overview

The Containers module supports deploying containers to one or more remote hosts via SSH. The system uses intelligent compose command detection with **Podman-first priority**:

1. **podman compose** (preferred for rootless, systemd integration)
2. **docker compose** (v2, plugin-based)
3. **docker-compose** (v1, standalone)

If detection fails, it falls back to the host's configured runtime.

Prerequisites

Remote Host Requirements

- SSH access with key-based authentication
- One of: Podman, Docker (with compose plugin), or docker-compose (v1)
- Network connectivity from the local machine

Local Machine Requirements

- SSH client with keys configured
- SSH agent running (optional, for key management)

Configuration

Environment Variables

Create a `.env` file in your project root or set environment variables:

```
# Enable remote distribution
CONTAINERS_REMOTE_ENABLED=true

# Scheduler strategy: resource_aware, round_robin, affinity,
# spread, bin_pack
CONTAINERS_REMOTE_SCHEDULER=resource_aware

# Define remote hosts (numbered 1, 2, 3, ...)
CONTAINERS_REMOTE_HOST_1_NAME=thinker
CONTAINERS_REMOTE_HOST_1_ADDRESS=thinker.local
CONTAINERS_REMOTE_HOST_1_USER=deploy
CONTAINERS_REMOTE_HOST_1_PORT=22
CONTAINERS_REMOTE_HOST_1_RUNTIME=podman
CONTAINERS_REMOTE_HOST_1_LABELS=gpu=false,arch=amd64

# Additional hosts
CONTAINERS_REMOTE_HOST_2_NAME=gpu-server
CONTAINERS_REMOTE_HOST_2_ADDRESS=192.168.1.100
CONTAINERS_REMOTE_HOST_2_RUNTIME=docker
CONTAINERS_REMOTE_HOST_2_LABELS=gpu=true,arch=amd64
```

Configuration File (Containers/.env)

Alternatively, create `Containers/.env`:

```
# Remote distribution
CONTAINERS_REMOTE_ENABLED=true

# Host 1: Podman host
CONTAINERS_REMOTE_HOST_1_NAME=thinker
CONTAINERS_REMOTE_HOST_1_ADDRESS=thinker.local
CONTAINERS_REMOTE_HOST_1_RUNTIME=podman

# Host 2: Docker host
CONTAINERS_REMOTE_HOST_2_NAME=prod-server
CONTAINERS_REMOTE_HOST_2_ADDRESS=prod.example.com
CONTAINERS_REMOTE_HOST_2_RUNTIME=docker
```

SSH Authentication

Key-Based Authentication (Recommended)

1. Generate an SSH key pair:

```
ssh-keygen -t ed25519 -C "deploy@yourdomain.com"
```

2. Copy the public key to the remote host:

```
ssh-copy-id -i ~/.ssh/id_ed25519.pub deploy@thinker.local
```

3. Add the key to SSH agent:

```
eval "$(ssh-agent -s)"  
ssh-add ~/.ssh/id_ed25519
```

SSH Config (~/.ssh/config)

```
Host thinker.local  
  HostName thinker.local  
  User deploy  
  IdentityFile ~/.ssh/id_ed25519  
  ControlMaster auto  
  ControlPath ~/.ssh/sockets/%r@%h-%p  
  ControlPersist 600
```

Compose Command Detection

The system automatically detects the best compose command on each remote host:

Detection Priority

Priority	Command	Notes
1	podman compose	Rootless, systemd integration
2	docker compose	Docker v2 plugin
3	docker-compose	Legacy v1 standalone

Manual Override

Force a specific compose command:

```
orch := NewRemoteComposeOrchestrator(  
  host, executor, logger,
```

```
    WithComposeCommand("podman compose"),
)
```

Detection Caching

Results are cached per host. Clear cache if runtime changes:

```
detector.ClearCache("thinker") // Clear specific host
detector.ClearCache("")         // Clear all hosts
```

Programmatic Usage

Basic Remote Deployment

```
package main

import (
    "context"
    "digital.vasic.containers/pkg/remote"
    "digital.vasic.containers/pkg/compose"
    "digital.vasic.containers/pkg/logging"
)

func main() {
    ctx := context.Background()
    logger := logging.NewDefaultLogger()

    // Create SSH executor
    executor := remote.NewSSHExecutor(logger)

    // Define remote host
    host := remote.RemoteHost{
        Name:     "thinker",
        Address:  "thinker.local",
        User:     "deploy",
        Runtime:  "podman",
    }

    // Create compose orchestrator (auto-detects compose command)
    orch := remote.NewRemoteComposeOrchestrator(host, executor,
        logger)

    // Deploy
    project := compose.ComposeProject{
        File: "/path/to/docker-compose.yml",
        Name: "myapp",
    }

    if err := orch.Up(ctx, project); err != nil {
```

```

        panic(err)
    }
}

```

Copy and Deploy

```

// Copy compose file to remote host
err := executor.CopyFile(ctx, host,
    "./docker-compose.yml",
    "/home/deploy/apps/myapp/docker-compose.yml",
)
if err != nil {
    panic(err)
}

// Deploy from copied file
project := compose.ComposeProject{
    File: "/home/deploy/apps/myapp/docker-compose.yml",
}
err = orch.Up(ctx, project)

```

Check Status

```

statuses, err := orch.Status(ctx, project)
for _, s := range statuses {
    fmt.Printf("%s: %s (health: %s)\n", s.Name, s.State, s.Health)
}

```

Tear Down

```

err := orch.Down(ctx, project)

```

Resource-Aware Scheduling

When multiple hosts are configured, the scheduler distributes containers based on available resources:

Scoring Formula

Resource	Weight
CPU	40%
Memory	40%
Disk	10%
Network	10%

Scheduling Strategies

Strategy	Description
resource_aware	Score-based, picks host with most available resources
round_robin	Distribute evenly across all hosts
affinity	Use labels to group containers on specific hosts
spread	Maximize distribution across hosts
bin_pack	Fill hosts before using new ones

Label-Based Affinity

```
# Host with GPU
CONTAINERS_REMOTE_HOST_1_LABELS=gpu=true,arch=amd64

# Host with high memory
CONTAINERS_REMOTE_HOST_2_LABELS=memory=high,arch=amd64

// Schedule on GPU host
project.Labels = map[string]string{"gpu": "required"}
```

Health Checking

Local vs Remote Services

When `CONTAINERS_REMOTE_ENABLED=true`, health checks target remote hosts:

```
// Health check against remote host
healthCheck := boot.TCPCheck("thinker.local", 5432)
```

Configure service as remote:

```
# In root .env (application config)
SVC_POSTGRESQL_REMOTE=true
SVC_REDIS_REMOTE=true
```

BootManager Integration

```
manager := boot.NewBootManager(
    boot.WithContainerAdapter(adapter),
    boot.WithHealthCheckRetries(5),
)

// Services are automatically health-checked against remote hosts
manager.AddService("postgresql", boot.ServiceConfig{
    HealthCheck: boot.TCPCheck("thinker.local", 5432),
})
```

```
    Required:    true,  
  })
```

Troubleshooting

"no compose command found"

Cause: Remote host has no compatible compose command installed.

Solution: Install Podman or Docker with compose plugin:

```
# Podman (recommended)  
sudo dnf install podman  
  
# Docker with compose plugin  
sudo apt install docker-compose-plugin
```

"connection refused"

Cause: SSH connection cannot be established.

Solutions:

1. Verify SSH key is added to agent: `ssh-add -l`
2. Test manual connection: `ssh deploy@thinker.local`
3. Check firewall allows SSH (port 22)

"exit 1: service not found"

Cause: Compose file references non-existent build contexts or services.

Solution:

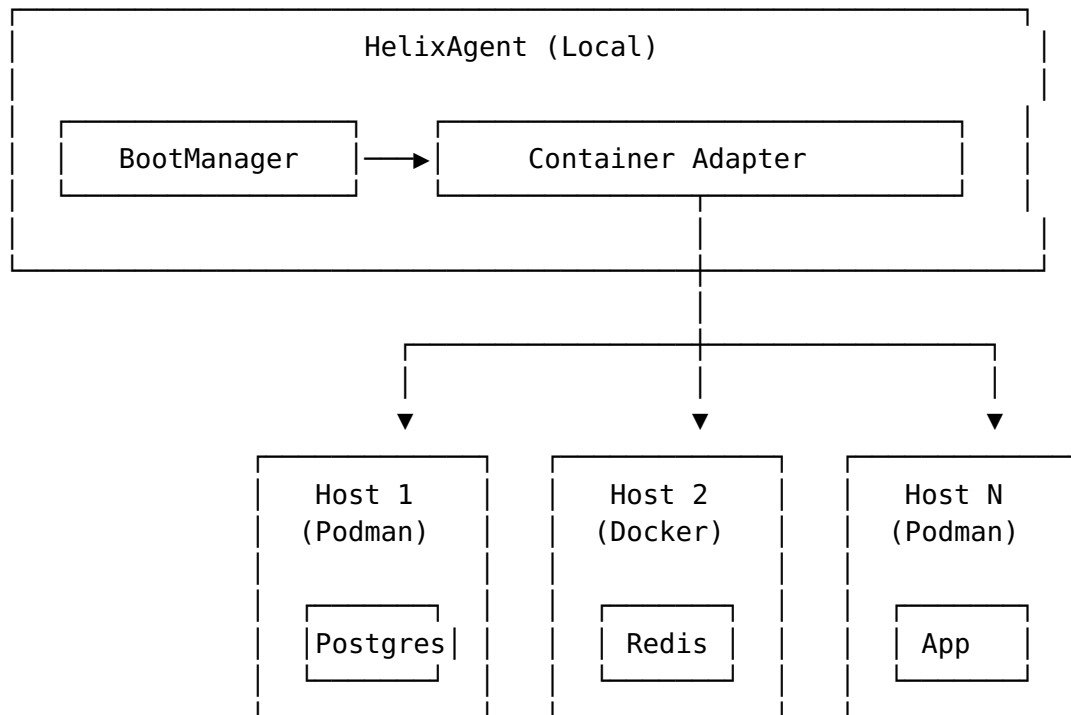
1. Ensure compose file exists on remote host
2. Use CopyFile to transfer compose files
3. Check build context paths are relative

Mixed Local/Remote Containers

Limitation: Current design requires ALL containers on remote OR ALL local.

Workaround: Use separate HelixAgent instances for mixed deployments.

Architecture



Best Practices

1. **Use Podman when possible:** Rootless, better systemd integration
2. **Enable SSH ControlMaster:** Faster repeated connections
3. **Configure resource labels:** Enable intelligent scheduling
4. **Health check all services:** Ensure reliability
5. **Use .env files:** Keep configuration out of code
6. **Monitor resource usage:** Prevent host overload

Last Updated: February 2026 **Version:** 2.0.0